

FINAL PROJECT
STRUKTUR DATA DAN PEMROGRAMAN BERORIENTASI OBJEK
2026

Kelompok 5:

5027251001	Evandra Raditya Fauzan
5027251035	Dea Chrisna Butarbutar
5027251053	Akbar Reyhan Fabian Susanto
5027251067	Muhammad Yusuf
5027251092	Naoval James Osamah

A. Deskripsi Masalah

a. Latar Belakang

Dalam dunia permainan video (video game), salah satu elemen terpenting yang menentukan kualitas pengalaman bermain adalah kemampuan karakter untuk menemukan jalur terbaik dalam sebuah peta. Peta permainan umumnya terdiri dari berbagai area yang memiliki karakteristik berbeda-beda, seperti tanah datar, hutan, pasir, air, lava, hingga tembok yang tidak bisa dilewati sama sekali. Setiap jenis medan (terrain) ini memiliki tingkat kesulitan atau biaya tersendiri untuk dilalui oleh karakter. Tanpa sistem pencarian jalur yang cerdas, karakter dalam game akan bergerak secara acak atau mengambil jalur yang tidak efisien, sehingga membuang sumber daya dan waktu. Di sinilah peran algoritma pathfinding menjadi sangat krusial. Algoritma yang baik harus mampu mempertimbangkan tidak hanya jarak, tetapi juga bobot dari setiap medan yang harus dilalui untuk menghasilkan jalur yang benar-benar optimal.

b. Rumusan Masalah

Berdasarkan latar belakang di atas, proyek ini bertujuan untuk menjawab permasalahan berikut:

1. Bagaimana merepresentasikan peta permainan dengan terrain berbobot menggunakan struktur data graph secara efisien?
2. Bagaimana mengimplementasikan algoritma A* untuk menemukan jalur terpendek dengan mempertimbangkan bobot terrain secara dinamis?
3. Bagaimana mengintegrasikan BFS sebagai alat eksplorasi map untuk menemukan semua checkpoint yang dapat dijangkau?
4. Bagaimana mengelola dan menampilkan checkpoint prioritas tinggi (berbahaya) secara efisien menggunakan struktur data tree?
5. Bagaimana memungkinkan perubahan bobot terrain secara dinamis dan menampilkan dampaknya terhadap jalur yang ditemukan?

c. Deskripsi Sistem

Sistem yang dibangun adalah sebuah aplikasi berbasis Java bernama Game Map Pathfinding AI. Aplikasi ini mensimulasikan peta permainan berupa grid 5x5 yang terdiri dari 25 checkpoint (node) dan 43 jalur (edge) yang menghubungkan

antar-checkpoint. Setiap checkpoint merepresentasikan satu lokasi dalam peta permainan, seperti Spawn Gate, Whisper Woods, Crystal Brook, dan sebagainya. Setiap jalur antar-checkpoint memiliki jenis terrain tertentu yang menentukan biaya traversal-nya. Terdapat enam jenis terrain yang didukung sistem:

- GROUND (biaya = 1): Medan paling mudah dilalui, merepresentasikan tanah biasa.
- FOREST (biaya = 2): Hutan yang sedikit lebih sulit dilalui.
- SAND (biaya = 3): Pasir yang memperlambat pergerakan karakter.
- WATER (biaya = 4): Air yang membutuhkan upaya lebih untuk diseberangi.
- LAVA (biaya = 5): Medan berbahaya dengan biaya traversal tinggi.
- WALL (biaya = Integer.MAX_VALUE): Tembok yang tidak bisa dilewati sama sekali (impassable).

Setiap checkpoint juga memiliki lima atribut tambahan yang memperkaya data dalam sistem, yaitu: zone (wilayah checkpoint), difficulty (tingkat kesulitan), loot (item yang bisa diambil), dangerScore (tingkat bahaya), dan blocked (apakah checkpoint dapat diakses atau tidak). Salah satu checkpoint khusus, yaitu C3 (Forbidden Core), memiliki status blocked = true sehingga tidak dapat dijadikan titik awal, tujuan, maupun jalur transit dalam pencarian rute.

d. Fitur Utama Sistem

- Input titik awal dan tujuan untuk pencarian jalur menggunakan A*.
- Menampilkan jalur hasil pencarian beserta total cost yang dibutuhkan.
- Eksplorasi seluruh map dari satu titik menggunakan BFS.
- Menampilkan area yang tidak dapat dilewati (checkpoint blocked dan route WALL).
- Mengubah bobot terrain secara dinamis dan membandingkan rute sebelum dan sesudah perubahan.
- Menampilkan ranking checkpoint berdasarkan danger score menggunakan struktur Max-Heap.
- Operasi CRUD lengkap untuk checkpoint dan route (insert, search, update, delete).
- Perbandingan efisiensi A* dengan Manhattan heuristic versus A* tanpa heuristic (setara Dijkstra).
- Penyimpanan perubahan dataset secara persisten ke file teks.

B. Dataset

a. Terrain Weights

Terrain_Type	Weight
GROUND	1
FOREST	2

SAND	3
WATER	4
LAVA	5
WALL	2147483647

b. Checkpoints

ID	name	pos_x	pos_y	zone	diff	loot	danger	blocked
A1	Spawn Gate	0	0	North	1	Potion	2	FALSE
A2	Whisper Woods	1	0	North	2	Herbs	3	FALSE
A3	Crystal Brook	2	0	North	2	Gem	2	FALSE
A4	Sand Watch	3	0	North	3	Map	4	FALSE
A5	Ember Ridge	4	0	North	4	Ore	6	FALSE
A6	Naoval's Garden	5	0	North	3	Durian	90	FALSE
B1	Iron Camp	0	1	West	2	Arrow	3	FALSE
B2	Moss Tunnel	1	1	West	3	Key	5	FALSE
B3	Ruined Bridge	2	1	Central	3	Relic	6	FALSE
B4	Dune Pass	3	1	East	3	Scroll	4	FALSE
B5	Lava Mouth	4	1	East	5	Core	8	FALSE
C1	Old Dock	0	2	West	2	Fish	2	FALSE
C2	Silent Marsh	1	2	Central	4	Pearl	6	FALSE
C3	Forbidden Core	2	2	Central	5	Artifact	9	TRUE
C4	Sunken Vault	3	2	East	4	Coin	7	FALSE
C5	Molten Stair	4	2	East	5	Crystal	8	FALSE
D1	Pine Shelter	0	3	South	1	Wood	2	FALSE
D2	Mist Crossing	1	3	South	2	Herb	3	FALSE
D3	Ancient Plaza	2	3	Central	3	Rune	5	FALSE
D4	Broken Tower	3	3	South	4	Gear	6	FALSE

D5	Ash Canyon	4	3	East	5	Ember	7	FALSE
E1	Harbor Exit	0	4	South	1	Rope	1	FALSE
E2	Green Hollow	1	4	South	2	Fruit	2	FALSE
E3	Watch Post	2	4	South	2	Badge	3	FALSE
E4	Shattered Gate	3	4	South	3	Medal	4	FALSE
E5	Final Beacon	4	4	South	4	Crown	5	FALSE

c. Routes

from	to	terrain	distance
A1	A2	GROUND	1
A1	A6	LAVA	54
A1	B1	GROUND	1
A2	A3	FOREST	1
A2	B2	FOREST	1
A3	A4	SAND	1
A3	B3	GROUND	1
A4	A5	LAVA	1
A4	B4	SAND	1
A5	B5	LAVA	1
A6	E2	LAVA	6
B1	B2	GROUND	1
B1	C1	GROUND	1
B2	B3	WATER	1
B2	C2	WATER	1
B3	B4	SAND	1
B3	C3	WALL	1

B4	B5	LAVA	1
B4	C4	WATER	1
B5	C5	LAVA	1
C1	C2	WATER	1
C1	D1	FOREST	1
C2	C3	WALL	1
C2	D2	GROUND	1
C3	C4	WALL	1
C3	D3	WALL	1
C4	C5	LAVA	1
C4	D4	SAND	1
C5	D5	LAVA	1
D1	D2	GROUND	1
D1	E1	GROUND	1
D2	D3	FOREST	1
D2	E2	GROUND	1
D3	D4	GROUND	1
D3	E3	FOREST	1
D4	D5	SAND	1
D4	E4	GROUND	1
D5	E5	SAND	1
E1	E2	GROUND	1
E2	E3	GROUND	1
E3	E4	FOREST	1
E4	E5	GROUND	1

C. Struktur Graph yang digunakan

a. Jenis Graph

Sistem ini menggunakan Weighted Undirected Graph sebagai representasi utama peta permainan. Disebut weighted karena setiap edge memiliki bobot yang ditentukan oleh jenis terrain yang dilaluinya. Disebut undirected karena setiap jalur antara dua checkpoint bersifat dua arah, artinya karakter dapat bergerak dari A ke B maupun dari B ke A dengan biaya yang sama.

Alasan memilih undirected graph, dalam konteks peta permainan ini, jalur antar-lokasi bersifat simetris. Jika seorang karakter bisa berjalan dari Spawn Gate menuju Iron Camp, maka secara logis ia juga bisa kembali dari Iron Camp menuju Spawn Gate dengan biaya yang sama. Hal ini berbeda dengan, misalnya, sistem jalan satu arah atau aliran air yang hanya mengalir satu arah.

b. Representasi Adjacency List

Graph direpresentasikan menggunakan Adjacency List yang diimplementasikan dalam kelas WeightedGraph menggunakan struktur data `LinkedHashMap<String, List<RouteEdge>>`.

Cara kerjanya adalah sebagai berikut:

- Setiap checkpoint (node) menjadi key dalam LinkedHashMap, dengan value berupa List yang berisi semua RouteEdge yang terhubung ke checkpoint tersebut.
- Saat sebuah route (edge) ditambahkan via `addRoute()`, route tersebut dimasukkan ke dalam list milik KEDUA endpoint, sehingga graph bersifat undirected.
- Penggunaan LinkedHashMap (bukan HashMap biasa) memastikan urutan checkpoint tampil sesuai urutan penyisipan dari dataset, sehingga output lebih mudah dibaca.

Alasan memilih Adjacency List dibanding Adjacency Matrix: Graph peta ini bersifat sparse (jarang terhubung). Dari kemungkinan $25 \times 24 / 2 = 300$ pasangan node, hanya 43 edge yang ada. Adjacency Matrix akan mengalokasikan memori $O(V^2) = O(625)$ meskipun sebagian besar sel bernilai nol. Adjacency List hanya membutuhkan memori $O(V + E)$, atau setara dengan $25 + 43 = 68$ elemen yang disimpan, jauh lebih hemat untuk graph sparse seperti ini.

c. Penanganan Blocked

Setiap edge dalam graph direpresentasikan oleh objek RouteEdge yang menyimpan informasi lengkap tentang suatu jalur, meliputi:

- `fromId`: ID checkpoint asal
- `told`: ID checkpoint tujuan
- `terrainType`: Jenis terrain (GROUND, FOREST, SAND, WATER, LAVA, WALL)

- baseCost: Biaya dasar dari dataset (umumnya 1, kecuali rute khusus seperti A1→A6 dengan baseCost = 54, sehingga total traversal cost = 54 × bobot terrain LAVA)

Biaya traversal aktual dihitung secara dinamis menggunakan method `getTraversalCost(terrainWeights)`, yaitu dengan mengalikan baseCost dengan bobot terrain yang aktif saat itu. Desain ini memungkinkan pengguna mengubah bobot terrain secara real-time dan langsung melihat dampaknya pada jalur yang ditemukan.

d. Penanganan Blocked Area

Sistem mendukung dua mekanisme untuk membatasi area yang tidak bisa diakses:

- Checkpoint blocked (`isBlocked = true`): Node seperti C3 (Forbidden Core) tidak bisa dijadikan titik awal, tujuan, maupun titik transit dalam BFS dan A*. Sistem memeriksa flag ini sebelum memproses setiap node.
- Route WALL (`isImpassable = true`): Edge dengan terrain WALL memiliki bobot `Integer.MAX_VALUE` dan diabaikan dalam traversal algoritma. Contohnya adalah jalur B3→C3, C2→C3, C3→C4, dan C3→D3 yang semuanya bertipe WALL, membentuk "dinding" di sekitar C3.

e. Statistik Graph

Parameter	Nilai
Jumlah node (checkpoint)	25 node
Jumlah edge (route)	43 edge
Jenis graph	Weighted Undirected Graph
Representasi	Adjacency List (LinkedHashMap)
Node blocked	1 node (C3 – Forbidden Core)
Edge impassable (WALL)	4 edge (B3↔C3, C2↔C3, C3↔C4, C3↔D3)
Jenis terrain	6 jenis (GROUND, FOREST, SAND, WATER, LAVA, WALL)

D. Struktur Tree yang digunakan

Pada project Game Map Pathfinding AI, struktur tree yang digunakan adalah Binary Heap. Binary Heap merupakan struktur data berbasis binary tree yang digunakan untuk menyimpan data berdasarkan prioritas. Binary Heap memiliki bentuk seperti

complete binary tree, yaitu setiap level terisi penuh kecuali mungkin level terakhir, dan pengisian node dilakukan dari kiri ke kanan.

Walaupun secara konsep berbentuk tree, Binary Heap umumnya direpresentasikan menggunakan array atau list agar lebih efisien. Dalam project ini, Binary Heap digunakan untuk membantu proses pencarian jalur dan pengelolaan prioritas checkpoint. Terdapat dua jenis heap yang digunakan, yaitu Min Heap dan Max Heap.

1. Min Heap

Min Heap adalah jenis Binary Heap yang menempatkan elemen dengan nilai prioritas terkecil pada posisi paling atas atau root. Dalam project ini, Min Heap digunakan pada proses pencarian jalur menggunakan algoritma A*. Setiap checkpoint kandidat memiliki nilai prioritas, dan checkpoint dengan nilai prioritas terkecil akan dipilih untuk diproses terlebih dahulu.

Nilai prioritas pada A* dihitung dengan rumus: $f(n) = g(n) + h(n)$

Keterangan:

$g(n)$ = cost dari titik awal ke checkpoint saat ini

$h(n)$ = estimasi cost dari checkpoint saat ini ke tujuan

$f(n)$ = nilai prioritas total

Semakin kecil nilai $f(n)$, semakin tinggi prioritas checkpoint untuk diproses.

Alasan Min Heap digunakan adalah karena algoritma A* membutuhkan proses pengambilan node dengan prioritas terkecil secara berulang. Dengan Min Heap, proses tersebut menjadi lebih efisien dibandingkan harus mencari nilai terkecil secara manual dari seluruh kandidat.

2. Max Heap

Max Heap adalah jenis Binary Heap yang menempatkan elemen dengan nilai prioritas terbesar pada posisi paling atas atau root. Dalam project ini, Max Heap digunakan untuk menentukan checkpoint dengan tingkat bahaya tertinggi. Setiap checkpoint memiliki nilai danger score, sehingga checkpoint dengan danger score paling besar akan memiliki prioritas tertinggi.

Contoh konsep:

Checkpoint A = danger score 2

Checkpoint B = danger score 5

Checkpoint C = danger score 9

Dalam Max Heap, checkpoint C akan berada di posisi teratas karena memiliki danger score terbesar. Alasan Max Heap digunakan adalah karena project memiliki fitur untuk menampilkan checkpoint yang paling berbahaya. Dengan Max Heap, checkpoint dengan danger score tertinggi dapat diakses dengan cepat tanpa harus mengurutkan seluruh data setiap kali dibutuhkan.

E. Algoritma yang digunakan

1. Breadth-First Search

Breadth-First Search atau BFS adalah algoritma penelusuran graph yang bekerja secara melebar. BFS memulai pencarian dari satu titik awal, lalu mengunjungi semua tetangga langsung dari titik tersebut sebelum melanjutkan ke titik yang lebih jauh.

Dalam project ini, BFS digunakan untuk melakukan eksplorasi map dari checkpoint tertentu. BFS membantu menampilkan urutan checkpoint yang dapat dijangkau dari titik awal.

Alur umum BFS:

1. Mulai dari checkpoint awal.
2. Masukkan checkpoint awal ke antrean.
3. Tandai checkpoint awal sebagai sudah dikunjungi.
4. Ambil checkpoint dari antrean.
5. Periksa semua checkpoint tetangganya.
6. Jika tetangga belum dikunjungi dan bisa dilewati, masukkan ke antrean.
7. Ulangi sampai tidak ada checkpoint yang tersisa di antrean.

BFS pada project ini tidak melewati checkpoint yang terblokir dan tidak melewati route dengan terrain WALL. Alasan BFS digunakan adalah karena BFS cocok untuk eksplorasi map. BFS dapat menunjukkan area mana saja yang dapat dicapai dari checkpoint awal. Namun, BFS tidak digunakan untuk mencari jalur termurah karena BFS tidak memperhitungkan bobot terrain.

2. A* Pathfinding

A* adalah algoritma pencarian jalur yang digunakan untuk mencari rute terbaik dari titik awal menuju titik tujuan. A* mempertimbangkan dua nilai utama:

$g(n)$ = cost yang sudah ditempuh dari titik awal ke checkpoint saat ini

$h(n)$ = estimasi cost dari checkpoint saat ini ke tujuan

Kedua nilai tersebut dijumlahkan menjadi:

$$f(n) = g(n) + h(n)$$

Checkpoint dengan nilai $f(n)$ paling kecil akan diproses terlebih dahulu. Dalam project ini, A* digunakan untuk mencari jalur terbaik pada map game yang memiliki berbagai jenis terrain. Setiap terrain memiliki bobot berbeda, sehingga jalur terbaik tidak hanya ditentukan oleh jumlah checkpoint yang dilewati, tetapi oleh total cost terkecil.

Contoh:

Jalur 1 melewati sedikit checkpoint, tetapi melalui terrain LAVA.

Jalur 2 melewati lebih banyak checkpoint, tetapi melalui terrain GROUND.

Dalam kondisi tersebut, jalur kedua bisa saja dipilih jika total cost-nya lebih kecil. Alasan A* digunakan adalah karena algoritma ini cocok untuk pathfinding pada game map. A* lebih terarah dibanding pencarian biasa karena menggunakan heuristic untuk memperkirakan jarak menuju tujuan.

3. Heuristic Manhattan Distance

Heuristic yang digunakan pada A* adalah Manhattan Distance. Manhattan Distance menghitung estimasi jarak berdasarkan selisih koordinat horizontal dan vertikal antara checkpoint saat ini dan checkpoint tujuan.

Rumus Manhattan Distance:

$$h(n) = |x_1 - x_2| + |y_1 - y_2|$$

Contoh:

Checkpoint awal berada di (0, 0)

Checkpoint tujuan berada di (3, 2)

$$h(n) = |0 - 3| + |0 - 2|$$

$$h(n) = 3 + 2$$

$$h(n) = 5$$

Alasan Manhattan Distance digunakan adalah karena checkpoint pada map memiliki koordinat x dan y. Heuristic ini cocok untuk map berbasis grid dan perhitungannya sederhana. Dengan heuristic, A* dapat lebih cepat mengarah ke tujuan karena tidak hanya mempertimbangkan cost yang sudah ditempuh, tetapi juga estimasi posisi tujuan.

4. Pencarian Tanpa Heuristic

Selain menggunakan heuristic, project ini juga menyediakan pencarian tanpa heuristic. Pada mode ini, nilai heuristic dianggap nol.

$$h(n) = 0$$

Sehingga:

$$f(n) = g(n)$$

Dengan kondisi tersebut, pencarian hanya mempertimbangkan cost dari titik awal ke checkpoint saat ini. Secara konsep, cara ini mirip dengan algoritma Dijkstra. Alasan pencarian tanpa heuristic digunakan adalah sebagai pembandingan terhadap A* dengan heuristic. Perbandingan ini dapat menunjukkan apakah heuristic membantu mengurangi jumlah checkpoint yang diproses. Jika A* dengan heuristic memproses lebih sedikit checkpoint dibanding tanpa heuristic, maka heuristic terbukti membantu meningkatkan efisiensi pencarian.

F. Design Decision Log

Bagian ini mendokumentasikan minimal lima keputusan desain penting yang diambil selama pengembangan sistem, beserta alternatif yang dipertimbangkan, alasan pemilihan, dan risiko atau kelemahan dari keputusan tersebut.

No	Keputusan	Alternatif	Alasan Memilih	Risiko/Kelemahan
1	Menggunakan Adjacency List (LinkedHashMap<String, List<RouteEdge>>) sebagai representasi graph	Adjacency Matrix (int[][] atau boolean[][])	Graph peta bersifat sparse: hanya 43 edge dari kemungkinan 300. Adjacency List hemat memori $O(V+E)$ dibanding $O(V^2)$ untuk matrix. LinkedHashMap juga menjaga urutan insertion sehingga output lebih konsisten.	Pengecekan apakah dua node terhubung (edge existence) membutuhkan $O(\text{degree})$ bukan $O(1)$ seperti matrix. Namun, untuk kasus A* dan BFS yang hanya butuh iterasi tetangga, ini tidak berpengaruh signifikan.
2	Menggunakan BinaryMinHeap kustom (bukan PriorityQueue bawaan Java) sebagai open list pada algoritma A*	java.util.PriorityQueue<FrontierNode>	Implementasi kustom memenuhi syarat proyek yang melarang penggunaan library graph/tree siap pakai untuk algoritma utama. BinaryMinHeap yang diimplementasikan sendiri juga memberikan pemahaman mendalam tentang cara kerja heap dan memungkinkan kustomisasi komparator sesuai kebutuhan.	Memerlukan pengujian lebih teliti untuk memastikan correctness, terutama pada operasi bubbleUp dan bubbleDown. Juga sedikit lebih verbose dibanding satu baris PriorityQueue.
3	Menggunakan heuristic Manhattan Distance yang di-scale dengan bobot terrain minimum sebagai heuristic A*	Euclidean Distance, atau heuristic konstan nol (zero heuristic = Dijkstra)	Manhattan Distance sesuai untuk grid-based map di mana pergerakan hanya dilakukan secara horizontal dan vertikal. Dikalikan dengan minWeight (bobot terrain terkecil yang bisa dilalui) agar	Jika bobot terrain diubah drastis oleh pengguna, nilai minWeight berubah dan heuristic menjadi kurang informatif, menyebabkan lebih banyak node yang diekspansi. Namun admissibility tetap terjaga.

			heuristic tetap admissible, yaitu tidak pernah melebihi biaya aktual, sehingga A* tetap menjamin jalur optimal.	
4	Menggunakan CheckpointMaxHeap kustom berbasis dangerScore untuk menampilkan ranking checkpoint berbahaya	Sorting list setiap kali diperlukan (Collections.sort), atau TreeSet	Max-Heap memungkinkan akses ke elemen dengan dangerScore tertinggi dalam $O(1)$ via peek(), dan ekstraksi top-K dalam $O(K \log N)$ tanpa mengubah heap asli (menggunakan salinan). Ini jauh lebih efisien dari sorting $O(N \log N)$ yang dilakukan berulang. Heap juga diimplementasikan dari scratch sesuai ketentuan proyek.	Heap tidak mendukung pencarian elemen arbitrary secara efisien ($O(N)$). Juga, setiap kali ada perubahan checkpoint (insert/update/delete), seluruh heap harus di-rebuild via heapify $O(N)$. Desain ini dipilih karena frekuensi perubahan data jauh lebih jarang dari operasi query.
5	Bobot traversal dihitung secara dinamis ($\text{baseCost} \times \text{terrainWeight}$) bukan disimpan statik di edge	Menyimpan effective cost langsung di RouteEdge saat dataset dibaca	Pemisahan antara baseCost (tetap dari dataset) dan terrainWeight (dapat diubah pengguna) memungkinkan fitur 'ubah bobot terrain dan tampilkan perubahan rute' berjalan tanpa perlu memodifikasi struktur graph. Cukup update EnumMap terrainWeights dan semua kalkulasi cost otomatis merefleksikan perubahan.	Setiap traversal edge memerlukan satu lookup ke EnumMap terrainWeights. Meskipun $O(1)$, ini sedikit lebih lambat dibanding akses field langsung. Untuk scale dataset 25 node, overhead ini tidak signifikan.
6	Menggunakan flag isBlocked pada Checkpoint dan	Menggunakan satu mekanisme saja, misalnya hanya	Dua mekanisme melayani tujuan berbeda: isBlocked	Menambah kompleksitas validasi karena setiap

	isImpassable pada TerrainType (WALL) sebagai dua mekanisme terpisah untuk area yang tidak bisa dilewati	weight = Integer.MAX_VALUE untuk semua kasus	menutup seluruh node (tidak bisa jadi start, end, maupun transit), sementara WALL menutup edge spesifik. Ini memungkinkan pemodelan yang lebih akurat, misalnya C3 (Forbidden Core) benar-benar tidak bisa diakses dari arah manapun.	algoritma (BFS, A*) harus memeriksa dua kondisi. Risiko bug jika salah satu pengecekan terlewat pada iterasi pengembangan.
7	Dataset disimpan dalam file teks terstruktur (.txt) dengan format section-based ([TERRAIN_WEIGHTS], [CHECKPOINTS], [ROUTES])	Database relasional (SQLite), JSON, atau XML	Format teks polos mudah dibaca dan diedit secara manual tanpa tools tambahan. Parsing sederhana dengan String.split() sudah cukup. Tidak memerlukan dependency eksternal, sesuai prinsip kesederhanaan untuk proyek akademik. Format section-based juga membuat struktur data mudah dipahami saat presentasi.	Tidak mendukung query kompleks atau indexing. Seluruh dataset selalu dibaca dan ditulis ulang secara lengkap setiap kali ada perubahan. Untuk dataset kecil (25 node, 43 edge), ini tidak menjadi masalah performa.

Setiap keputusan di atas diambil berdasarkan pertimbangan antara efisiensi algoritma, keterbacaan kode, ketentuan proyek, dan kesesuaian dengan karakteristik dataset yang digunakan. Keputusan-keputusan ini juga dievaluasi dalam konteks kemampuan sistem untuk menangani perubahan dinamis seperti update bobot terrain dan penambahan checkpoint baru.

G. Tracing Manual

1. Tracing Proses Tree

- Representasi Heap dalam Array

Heap direpresentasikan menggunakan ArrayList. Hubungan antara parent dan child dihitung menggunakan rumus:

$$\text{left child} = 2i + 1$$

$$\text{right child} = 2i + 2$$

$$\text{parent} = (i - 1) / 2$$

Kondisi Awal Sebelum Heapify:

{ 'A1', 'A2', 'A3', 'A4', 'A5', 'A6', 'B1', 'B2', 'B3', 'B4', 'B5', 'C1', 'C2', 'C3', 'C4', 'C5', 'D1', 'D2', 'D3', 'D4', 'D5', 'E1', 'E2', 'E3', 'E4', 'E5' }

- **Manual Tracing Proses Heapify**

Jumlah checkpoint adalah 26, sehingga proses heapify dimulai dari index:

$$n / 2 - 1 = 26 / 2 - 1 = 12$$

Artinya, proses bubbleDown() dilakukan mulai dari index 12 sampai index 0.

Langkah	Index	Node yang Dicek	Proses
1	12	C2(6)	Tidak terjadi swap karena child tidak lebih besar.
2	11	C1(2)	Swap dengan E4(4), karena E4 memiliki danger score lebih tinggi.
3	10	B5(8)	Tidak terjadi swap.
4	9	B4(4)	Swap dengan D5(7).
5	8	B3(6)	Tidak terjadi swap.
6	7	B2(5)	Swap dengan C5(8).
7	6	B1(3)	Swap dengan C3(9).
8	5	A6(90)	Tidak terjadi swap karena A6 sudah memiliki nilai sangat besar.
9	4	A5(6)	Swap dengan B5(8).
10	3	A4(4)	Swap dengan C5(8).
11	2	A3(2)	Swap dengan A6(90), lalu bubble down dilanjutkan.
12	1	A2(3)	Swap dengan B5(8), lalu bubble down dilanjutkan.
13	0	A1(2)	Swap dengan A6(90), lalu bubble down dilanjutkan sampai posisi heap valid.

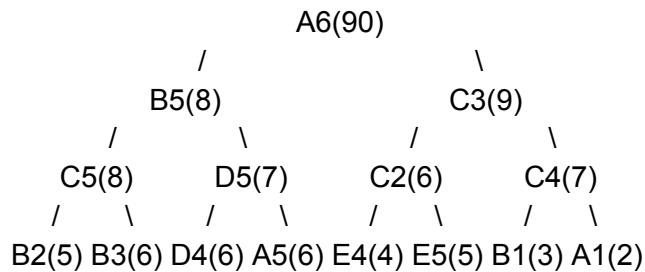
- **Hasil Akhir Heap**

Isi heap dalam bentuk array setelah proses heapify selesai:

{A6(90), B5(8), C3(9), C5(8), D5(7), C2(6), C4(7), B2(5), B3(6), D4(6), A5(6), E4(4), E5(5), B1(3), A1(2), A4(4), D1(2), D2(3), D3(5), A2(3), B4(4), E1(1), E2(2), E3(3), C1(2), A3(2)}

Dengan root heap: A6(90)

- **Bentuk Tree Setelah Heapify**



Meskipun C3(9) lebih besar daripada B5(8), struktur ini tetap valid karena keduanya adalah child dari A6(90). Dalam heap, yang penting adalah hubungan parent-child, bukan urutan antar sibling.

- **Manual Tracing Operasi Peek**

Operasi peek() digunakan untuk melihat checkpoint dengan prioritas tertinggi tanpa menghapusnya dari heap.

Karena root heap berada pada index 0, maka:

peek() = heap[0] = A6(90)

Langkah	Index	Node yang Dicek
1	0	A6(90)

- **Manual Tracing Operasi Top 5 Ranking**

Untuk menampilkan ranking checkpoint paling berbahaya, program menggunakan method toSortedList(limit). Method ini membuat salinan heap, lalu melakukan proses extractMax() berulang kali.

Pada setiap proses extractMax():

1. Root heap diambil sebagai nilai maksimum.
2. Elemen terakhir dipindahkan ke root.
3. Proses bubbleDown() dilakukan untuk mengembalikan sifat Max Heap.
4. Nilai maksimum dimasukkan ke list hasil ranking.

Urutan	Root yang Diambil	Keterangan
--------	-------------------	------------

1	A6(90)	Danger score terbesar, langsung keluar pertama.
2	C3(9)	Setelah A6 dihapus, C3 menjadi prioritas tertinggi.
3	B5(8)	Memiliki danger score 8 dan ID lebih kecil dibanding C5.
4	C5(8)	Danger score sama dengan B5, tetapi ID lebih besar, sehingga keluar setelah B5.
5	C4(7)	Setelah nilai 8 habis, checkpoint dengan nilai 7 menjadi prioritas berikutnya.

Maka hasil ranking top 5 adalah:

1. A6(90)
2. C3(9)
3. B5(8)
4. C5(8)
5. C4(7)

Checkpoint C3 tetap masuk ke heap walaupun statusnya blocked = true, karena heap ini dibuat berdasarkan seluruh checkpoint dan tidak memfilter checkpoint yang terblokir.

2. Tracing Proses Graph

- Tracing BFS dari Spawn Gate (A1)

BFS melakukan eksplorasi secara merata dengan FIFO, tanpa mempertimbangkan bobot terrain, mengabaikan adanya wall dan blocked node.

Langkah	Dequeue	Asal Node	Enqueue
1	A1	A1	A2, A6, B1
2	A2	A1	A3, B2
3	A6	A1	E2
4	B1	A1	C1
5	A3	A2	A4, B3
6	B2	A2	C2

7	E2	A6	D2, E1, E3
8	C1	B1	D1
9	A4	A3	A5, B4
10	B3	A3	- (C3 = WALL)
11	C2	B2	-
12	D2	C2	D3
13	E1	E2	-
14	E3	E2	E4
15	D1	C1	-
16	A5	A4	B5
17	B4	A4	C4
18	D3	D2	D4
19	E4	E3	E5
20	B5	A5	C5
21	C4	B4	-
22	D4	D3	D5
23	E5	E4	-
24	C5	B5	-
25	D5	D4	-

- **Tracing A***

A* memakai $f(n) = g(n) + h(n)$.

Dengan heuristic:

Langkah	Dequeue	g	h	f	Asal Node	Enqueue
1	A1	0	8	8	-	A2, A6, B1
2	A2	1	7	8	A1	A3, B2
3	B1	1	7	8	A1	B2, C1

4	B2	2	6	8	B1	B3, C2
5	C1	2	6	8	B1	D1
6	A3	3	6	9	A2	A4, B3
7	D1	4	5	9	C1	D2, E1
8	B3	4	5	9	A3	B4
9	D2	5	4	9	D1	D3, E2
10	E2	6	3	9	D2	A6, E3
11	E2	6	3	9	D2	A6, E3
12	E3	7	2	9	E2	E4
13	D3	7	3	10	D2	D4
14	D4	8	2	10	D3	C4, D5
15	E5	10	0	10	E4	-

Jalur: A1 - B1 - C1 - D1 - D2 - E2- E3- E4 - E5

Total cost: 10

Tanpa heuristic:

Langkah	Dequeue	g	h	f	Asal Node	Enqueue
1	A1	0	0	0	-	A2, A6, B1
2	A2	1	0	1	A1	A3, B2
3	B1	1	0	1	A1	B2, C1
4	B2	2	0	2	B1	B3, C2
5	C1	2	0	2	B1	D1
6	A3	3	0	3	A2	A4, B3
7	D1	4	0	4	C1	D2, E1
8	B3	4	0	4	A3	B4
9	D2	5	0	5	D1	D3, E2

10	E1	5	0	5	D1	-
11	C2	6	0	6	B2	-
12	A4	6	0	6	A3	A5
13	E2	6	0	6	D2	A6, E3
14	B4	7	0	7	B3	B5, C4
15	D3	7	0	7	D2	D4
16	E3	7	0	7	E2	E4
17	D4	8	0	8	D3	D5
18	E4	9	0	9	E3	E5
19	E5	10	0	10	E4	-

3. Tracing Edge Case

- Checkpoint Awal Tidak Valid

Contoh kasus:

Start = Z9

Goal = E5

Langkah	Kondisi	Respon Program
1	User memasukkan start Z9 dan goal E5.	Program menerima input.
2	Program mencari checkpoint Z9.	Checkpoint tidak ditemukan.
3	Start tidak valid.	Pencarian tidak dijalankan.
4	Program mengembalikan hasil.	Jalur tidak ditemukan.

Kesimpulan: Jika checkpoint awal tidak tersedia di map, algoritma tidak dijalankan karena tidak ada titik mulai yang valid untuk melakukan pencarian.

- Checkpoint Tujuan Tidak Valid

Contoh kasus:

Start = A1

Goal = Z9

Langkah	Kondisi	Respon Program
---------	---------	----------------

1	User memasukkan start A1 dan goal Z9.	Program menerima input.
2	Program memeriksa A1.	Checkpoint awal valid.
3	Program mencari Z9.	Checkpoint tujuan tidak ditemukan.
4	Goal tidak valid.	Pencarian dihentikan.
5	Program mengembalikan hasil.	Jalur tidak ditemukan.

Kesimpulan: Jika checkpoint tujuan tidak tersedia, program tidak memiliki target pencarian sehingga jalur tidak dapat ditemukan.

- Checkpoint Tujuan Terblokir

Contoh kasus:

Start = A1

Goal = C3

Pada dataset, C3 merupakan checkpoint yang terblokir.

Langkah	Kondisi	Respon Program
1	User memasukkan start A1 dan goal C3.	Program menerima input.
2	Program memeriksa A1.	Checkpoint awal valid.
3	Program memeriksa C3.	Checkpoint ditemukan.
4	Status C3 adalah blocked.	Goal dianggap tidak dapat diakses.
5	Program menghentikan pencarian.	Jalur tidak ditemukan.

Kesimpulan: Checkpoint tujuan yang blocked tidak dapat dijadikan target akhir karena area tersebut dianggap tidak dapat diakses.

- Route Bertipe WALL

Contoh kasus:

Route B3 -> C3 memiliki terrain WALL

Langkah	Kondisi	Respon Program
1	Program memeriksa route dari checkpoint saat ini.	Route-route tetangga dicek.

2	Program menemukan route bertipe WALL.	Route dianggap tidak dapat dilewati.
3	Route WALL diabaikan.	Program tidak memasukkan tujuan route tersebut sebagai kandidat.
4	Program lanjut ke route lain.	Hanya route valid yang diproses.

Kesimpulan: Route bertipe WALL tetap tersimpan sebagai bagian dari map, tetapi tidak digunakan sebagai jalur valid dalam BFS maupun A*.

- **Jalur Lebih Pendek Tetapi Cost Lebih Mahal**

Contoh kasus:

Jalur 1: A1 -> A6 -> E2 -> E3 -> E4 -> E5

Jalur 2: A1 -> B1 -> C1 -> D1 -> E1 -> E2 -> E3 -> E4 -> E5

Langkah	Kondisi	Respon Program
1	Program menemukan beberapa kemungkinan jalur.	Semua kandidat jalur dievaluasi.
2	Jalur 1 memiliki jumlah checkpoint lebih sedikit.	Program tetap menghitung total cost.
3	Jalur 1 melewati terrain berat seperti LAVA.	Total cost menjadi lebih besar.
4	Jalur 2 memiliki checkpoint lebih banyak.	Program tetap membandingkan cost.
5	Jalur 2 melewati terrain lebih ringan.	Total cost dapat menjadi lebih kecil.
6	Program memilih jalur dengan cost terkecil.	Jalur terbaik ditentukan dari total cost, bukan jumlah checkpoint.

Kesimpulan: Dalam weighted graph, jalur terbaik ditentukan berdasarkan total cost, bukan berdasarkan jumlah checkpoint yang dilewati. Karena itu, jalur yang lebih panjang bisa menjadi pilihan terbaik jika melewati terrain dengan bobot lebih ringan.

- **Tidak Ada Jalur Valid ke Tujuan**

Contoh kasus:

Start dan goal tersedia, tetapi semua jalur menuju goal terhalang WALL atau checkpoint blocked.

Langkah	Kondisi	Respon Program
1	Program memulai pencarian dari start.	Start valid.
2	Program memeriksa route yang tersedia.	Route valid diproses.
3	Route WALL ditemukan.	Route diabaikan.
4	Checkpoint blocked ditemukan.	Checkpoint diabaikan.
5	Semua kandidat habis diperiksa.	Goal tidak tercapai.
6	Program mengembalikan hasil.	Jalur tidak ditemukan.

Kesimpulan: Meskipun start dan goal tersedia di map, jalur tetap bisa tidak ditemukan jika semua kemungkinan route menuju tujuan tidak dapat dilewati.

- **Start dan Goal Sama**

Contoh kasus:

Start = A1

Goal = A1

Langkah	Kondisi	Respon Program
1	User memasukkan start dan goal yang sama.	Program menerima input.
2	Program memeriksa checkpoint tersebut.	Checkpoint valid dan tidak blocked.
3	Start sudah sama dengan goal.	Pencarian langsung selesai.
4	Program membentuk hasil jalur.	Jalur hanya berisi A1.
5	Program menghitung cost.	Total cost bernilai 0.

Kesimpulan: Jika start dan goal sama, jalur valid tetap ditemukan dengan total cost 0 karena karakter sudah berada di checkpoint tujuan.

H. Screenshot Hasil Program

1. Tampilan Menu

```
=====
FP Strukdat - Game Map Pathfinding AI
=====
Checkpoint: 26 | Route: 42 | Terrain types: 6 | Tree: Custom Max Heap | Algorithms: A*,
BFS
Menu:
1. Tampilkan semua checkpoint
2. Tampilkan struktur graph
3. Search checkpoint
4. Insert checkpoint
5. Update checkpoint
6. Delete checkpoint
7. Kelola route/edge
```

2. Menu 1: Tampilkan semua checkpoint

Pilih menu: 1

Daftar checkpoint (26):

```
- A1 | Spawn Gate | pos=(0,0) | zone=North | diff=1 | loot=Potion | danger=2 | blocked=false
- A2 | Whisper Woods | pos=(1,0) | zone=North | diff=2 | loot=Herbs | danger=3 | blocked=false
- A3 | Crystal Brook | pos=(2,0) | zone=North | diff=2 | loot=Gem | danger=2 | blocked=false
- A4 | Sand Watch | pos=(3,0) | zone=North | diff=3 | loot=Map | danger=4 | blocked=false
- A5 | Ember Ridge | pos=(4,0) | zone=North | diff=4 | loot=Ore | danger=6 | blocked=false
- A6 | Naoval's Garden | pos=(5,0) | zone=North | diff=3 | loot=Durian | danger=90 | blocked=false
- B1 | Iron Camp | pos=(0,1) | zone=West | diff=2 | loot=Arrow | danger=3 | blocked=false
- B2 | Moss Tunnel | pos=(1,1) | zone=West | diff=3 | loot=Key | danger=5 | blocked=false
- B3 | Ruined Bridge | pos=(2,1) | zone=Central | diff=3 | loot=Relic | danger=6 | blocked=false
- B4 | Dune Pass | pos=(3,1) | zone=East | diff=3 | loot=Scroll | danger=4 | blocked=false
- B5 | Lava Mouth | pos=(4,1) | zone=East | diff=5 | loot=Core | danger=8 | blocked=false
- C1 | Old Dock | pos=(0,2) | zone=West | diff=2 | loot=Fish | danger=2 | blocked=false
- C2 | Silent Marsh | pos=(1,2) | zone=Central | diff=4 | loot=Pearl | danger=6 | blocked=false
- C3 | Forbidden Core | pos=(2,2) | zone=Central | diff=5 | loot=Artifact | danger=9 | blocked=true
- C4 | Sunken Vault | pos=(3,2) | zone=East | diff=4 | loot=Coin | danger=7 | blocked=false
- C5 | Molten Stair | pos=(4,2) | zone=East | diff=5 | loot=Crystal | danger=8 | blocked=false
- D1 | Pine Shelter | pos=(0,3) | zone=South | diff=1 | loot=Wood | danger=2 | blocked=false
- D2 | Mist Crossing | pos=(1,3) | zone=South | diff=2 | loot=Herb | danger=3 | blocked=false
- D3 | Ancient Plaza | pos=(2,3) | zone=Central | diff=3 | loot=Rune | danger=5 | blocked=false
- D4 | Broken Tower | pos=(3,3) | zone=South | diff=4 | loot=Gear | danger=6 | blocked=false
- D5 | Ash Canyon | pos=(4,3) | zone=East | diff=5 | loot=Ember | danger=7 | blocked=false
- E1 | Harbor Exit | pos=(0,4) | zone=South | diff=1 | loot=Rope | danger=1 | blocked=false
- E2 | Green Hollow | pos=(1,4) | zone=South | diff=2 | loot=Fruit | danger=2 | blocked=false
- E3 | Watch Post | pos=(2,4) | zone=South | diff=2 | loot=Badge | danger=3 | blocked=false
- E4 | Shattered Gate | pos=(3,4) | zone=South | diff=3 | loot=Medal | danger=4 | blocked=false
- E5 | Final Beacon | pos=(4,4) | zone=South | diff=4 | loot=Crown | danger=5 | blocked=false
```

3. Menu 2: Tampilkan Struktur Graph

```
Pilih menu: 2
```

Adjacency List Graph

```
A1 -> A2(GROUND:1), A6(LAVA:270), B1(GROUND:1)
A2 -> A1(GROUND:1), A3(FOREST:2), B2(FOREST:2)
A3 -> A2(FOREST:2), A4(SAND:3), B3(GROUND:1)
A4 -> A3(SAND:3), A5(LAVA:5), B4(SAND:3)
A5 -> A4(LAVA:5), B5(LAVA:5)
A6 -> A1(LAVA:270), E2(LAVA:30)
B1 -> A1(GROUND:1), B2(GROUND:1), C1(GROUND:1)
B2 -> A2(FOREST:2), B1(GROUND:1), B3(WATER:4), C2(WATER:4)
B3 -> A3(GROUND:1), B2(WATER:4), B4(SAND:3), C3(WALL:X)
B4 -> A4(SAND:3), B3(SAND:3), B5(LAVA:5), C4(WATER:4)
B5 -> A5(LAVA:5), B4(LAVA:5), C5(LAVA:5)
C1 -> B1(GROUND:1), C2(WATER:4), D1(FOREST:2)
C2 -> B2(WATER:4), C1(WATER:4), C3(WALL:X), D2(GROUND:1)
C3 -> B3(WALL:X), C2(WALL:X), C4(WALL:X), D3(WALL:X)
C4 -> B4(WATER:4), C3(WALL:X), C5(LAVA:5), D4(SAND:3)
C5 -> B5(LAVA:5), C4(LAVA:5), D5(LAVA:5)
D1 -> C1(FOREST:2), D2(GROUND:1), E1(GROUND:1)
D2 -> C2(GROUND:1), D1(GROUND:1), D3(FOREST:2), E2(GROUND:1)
D3 -> C3(WALL:X), D2(FOREST:2), D4(GROUND:1), E3(FOREST:2)
D4 -> C4(SAND:3), D3(GROUND:1), D5(SAND:3), E4(GROUND:1)
D5 -> C5(LAVA:5), D4(SAND:3), E5(SAND:3)
E1 -> D1(GROUND:1), E2(GROUND:1)
E2 -> A6(LAVA:30), D2(GROUND:1), E1(GROUND:1), E3(GROUND:1)
E3 -> D3(FOREST:2), E2(GROUND:1), E4(FOREST:2)
E4 -> D4(GROUND:1), E3(FOREST:2), E5(GROUND:1)
E5 -> D5(SAND:3), E4(GROUND:1)
```

4. Menu 3: Search Checkpoint

```
Pilih menu: 3
```

```
Masukkan ID atau nama checkpoint: A3
```

```
Hasil pencarian:
```

```
- A3 | Crystal Brook | pos=(2,0) | zone=North | diff=2 | loot=Gem | danger=2 | blocked=false
```

```
Pilih menu: 3
```

```
Masukkan ID atau nama checkpoint: A7
```

```
Checkpoint tidak ditemukan.
```

5. Menu 4: Insert Checkpoint


```
Pilih menu: 4

Insert checkpoint baru
ID [baru]: B6
Nama: Base Camp
Koordinat X: 5
Koordinat Y: 5
Zone: South
Difficulty: 3
Loot: Elixir
Danger score: 4
Blocked (true/false): false
Checkpoint berhasil ditambahkan.
```

```
Daftar checkpoint (27):
- A1 | Spawn Gate | pos=(0,0) | zone=North | diff=1 | loot=Potion | danger=2 | blocked=false
- A2 | Whisper Woods | pos=(1,0) | zone=North | diff=2 | loot=Herbs | danger=3 | blocked=false
- A3 | Crystal Brook | pos=(2,0) | zone=North | diff=2 | loot=Gem | danger=2 | blocked=false
- A4 | Sand Watch | pos=(3,0) | zone=North | diff=3 | loot=Map | danger=4 | blocked=false
- A5 | Ember Ridge | pos=(4,0) | zone=North | diff=4 | loot=Ore | danger=6 | blocked=false
- A6 | Naoval's Garden | pos=(5,0) | zone=North | diff=3 | loot=Durian | danger=90 | blocked=false
- B1 | Iron Camp | pos=(0,1) | zone=West | diff=2 | loot=Arrow | danger=3 | blocked=false
- B2 | Moss Tunnel | pos=(1,1) | zone=West | diff=3 | loot=Key | danger=5 | blocked=false
- B3 | Ruined Bridge | pos=(2,1) | zone=Central | diff=3 | loot=Relic | danger=6 | blocked=false
- B4 | Dune Pass | pos=(3,1) | zone=East | diff=3 | loot=Scroll | danger=4 | blocked=false
- B5 | Lava Mouth | pos=(4,1) | zone=East | diff=5 | loot=Core | danger=8 | blocked=false
- C1 | Old Dock | pos=(0,2) | zone=West | diff=2 | loot=Fish | danger=2 | blocked=false
- C2 | Silent Marsh | pos=(1,2) | zone=Central | diff=4 | loot=Pearl | danger=6 | blocked=false
- C3 | Forbidden Core | pos=(2,2) | zone=Central | diff=5 | loot=Artifact | danger=9 | blocked=true
- C4 | Sunken Vault | pos=(3,2) | zone=East | diff=4 | loot=Coin | danger=7 | blocked=false
- C5 | Molten Stair | pos=(4,2) | zone=East | diff=5 | loot=Crystal | danger=8 | blocked=false
- D1 | Pine Shelter | pos=(0,3) | zone=South | diff=1 | loot=Wood | danger=2 | blocked=false
- D2 | Mist Crossing | pos=(1,3) | zone=South | diff=2 | loot=Herb | danger=3 | blocked=false
- D3 | Ancient Plaza | pos=(2,3) | zone=Central | diff=3 | loot=Rune | danger=5 | blocked=false
- D4 | Broken Tower | pos=(3,3) | zone=South | diff=4 | loot=Gear | danger=6 | blocked=false
- D5 | Ash Canyon | pos=(4,3) | zone=East | diff=5 | loot=Ember | danger=7 | blocked=false
- E1 | Harbor Exit | pos=(0,4) | zone=South | diff=1 | loot=Rope | danger=1 | blocked=false
- E2 | Green Hollow | pos=(1,4) | zone=South | diff=2 | loot=Fruit | danger=2 | blocked=false
- E3 | Watch Post | pos=(2,4) | zone=South | diff=2 | loot=Badge | danger=3 | blocked=false
- E4 | Shattered Gate | pos=(3,4) | zone=South | diff=3 | loot=Medal | danger=4 | blocked=false
- E5 | Final Beacon | pos=(4,4) | zone=South | diff=4 | loot=Crown | danger=5 | blocked=false
- B6 | Base Camp | pos=(5,5) | zone=South | diff=3 | loot=Elixir | danger=4 | blocked=false
```

6. Menu 5: Update Checkpoint

```
Pilih menu: 5

Masukkan ID checkpoint yang ingin diupdate: B6
Data lama: B6 | Base Camp | pos=(5,5) | zone=South | diff=3 | loot=Elixir | danger=4 | b
locked=false
ID [B6]: F1
Nama [Base Camp]: Hidden Sanctuary
Koordinat X [5]: 5
Koordinat Y [5]: 5
Zone [South]: South
Difficulty [3]: 2
Loot [Elixir]: Legendary Elixir
Danger score [4]: 2
Blocked [false] (true/false): false
Checkpoint berhasil diupdate.
```

7. Menu 6: Delete Checkpoint

```
Pilih menu: 6

Masukkan ID checkpoint yang ingin dihapus: F1
Checkpoint dan route terkait berhasil dihapus.
```

8. Menu 7: Kelola route/edge

1. Tambah route

```
Pilih menu: 7

Kelola route
1. Tambah route
2. Update route
3. Delete route
Pilih aksi: 1
Checkpoint asal: E5
Checkpoint tujuan: F1
Terrain (GROUND, FOREST, SAND, WATER, LAVA, WALL): GROUND
Base cost: 1
Route berhasil ditambahkan.
```

2. Update route

```
Pilih menu: 7

Kelola route
1. Tambah route
2. Update route
3. Delete route
Pilih aksi: 2
Masukkan ID checkpoint asal: E5
Masukkan ID checkpoint tujuan: F1
Data lama: E5 <-> F1 | terrain=GROUND | baseCost=1 | totalCost=1
Checkpoint asal [E5]: E5
Checkpoint tujuan [F1]: F1
Terrain [GROUND] (GROUND, FOREST, SAND, WATER, LAVA, WALL): FOREST
Base cost [1]: 2
Route berhasil diupdate.
```

3. Delete node

```
Pilih menu: 7

Kelola route
1. Tambah route
2. Update route
3. Delete route
Pilih aksi: 3
Masukkan ID checkpoint asal: E5
Masukkan ID checkpoint tujuan: F1
Route berhasil dihapus.
```

9. Menu 8: Tampilkan area tidak dapat dilewati

```
Pilih menu: 8

Checkpoint terblokir:
- C3 (Forbidden Core)
Route impassable:
- B3 <-> C3 (WALL)
- C2 <-> C3 (WALL)
- C3 <-> C4 (WALL)
- C3 <-> D3 (WALL)
```

10. Menu 9: Eksplorasi map dengan BFS

```
Pilih menu: 9

Masukkan checkpoint awal BFS: A1
Urutan eksplorasi BFS:
A1 -> A2 -> A6 -> B1 -> A3 -> B2 -> E2 -> C1 -> A4 -> B3 -> C2 -> D2 -> E1 -> E3 ->
D1 -> A5 -> B4 -> D3 -> E4 -> B5 -> C4 -> D4 -> E5 -> C5 -> D5
```

11. Menu 10: Cari jalur dengan A*

```
Pilih menu: 10

Masukkan checkpoint awal: A1
Masukkan checkpoint tujuan: E5
Jalur terbaik: A1 -> B1 -> C1 -> D1 -> E1 -> E2 -> E3 -> E4 -> E5
Total cost: 10
Node diekspansi: 16
Heuristic: Manhattan
```

12. Menu 11: Bandingkan heuristic A* vs tanpa heuristic

Pilih menu: 11

Masukkan checkpoint awal: A1

Masukkan checkpoint tujuan: E5

A* dengan Manhattan:

- Jalur: A1 -> B1 -> C1 -> D1 -> E1 -> E2 -> E3 -> E4 -> E5

- Total cost: 10

- Node diekspansi: 16

A* tanpa heuristic (setara Dijkstra):

- Jalur: A1 -> B1 -> C1 -> D1 -> E1 -> E2 -> E3 -> E4 -> E5

- Total cost: 10

- Node diekspansi: 19

Analisis HOTS:

Heuristic Manhattan mengurangi jumlah node yang diekspansi sebanyak 3 node sambil tetap menjaga cost optimal. Ini menunjukkan heuristic membantu mempercepat pencarian.

13. Menu 12: Ubah bobot terrain

Pilih menu: 12

Bobot terrain saat ini:

- GROUND = 1

- FOREST = 2

- SAND = 3

- WATER = 4

- LAVA = 5

- WALL = 2147483647

Masukkan checkpoint awal untuk simulasi perubahan rute: A1

Masukkan checkpoint tujuan untuk simulasi perubahan rute: A6

Terrain (GROUND, FOREST, SAND, WATER, LAVA, WALL): LAVA

Masukkan bobot baru terrain: 10

Bobot terrain berhasil diubah.

Perbandingan rute:

Sebelum perubahan:

- Jalur: A1 -> B1 -> C1 -> D1 -> D2 -> E2 -> A6

- Total cost: 36

- Node diekspansi: 25

Sesudah perubahan:

- Jalur: A1 -> B1 -> C1 -> D1 -> D2 -> E2 -> A6

- Total cost: 66

- Node diekspansi: 25

14. Menu 13: Tampilkan checkpoint prioritas heap

```
Pilih menu: 13
```

```
Checkpoint paling berbahaya (heap peek):
```

```
A6 | Naoval's Garden | pos=(5,0) | zone=North | diff=3 | loot=Durian | danger=90 | blocked=false
```

```
Top 5 checkpoint berdasarkan danger score:
```

```
1. A6 | Naoval's Garden | pos=(5,0) | zone=North | diff=3 | loot=Durian | danger=90 | blocked=false
```

```
2. C3 | Forbidden Core | pos=(2,2) | zone=Central | diff=5 | loot=Artifact | danger=9 | blocked=true
```

```
3. B5 | Lava Mouth | pos=(4,1) | zone=East | diff=5 | loot=Core | danger=8 | blocked=false
```

```
4. C5 | Molten Stair | pos=(4,2) | zone=East | diff=5 | loot=Crystal | danger=8 | blocked=false
```

```
5. C4 | Sunken Vault | pos=(3,2) | zone=East | diff=4 | loot=Coin | danger=7 | blocked=false
```

15. Menu 14: Simpan dataset

```
Pilih menu: 14
```

```
Dataset berhasil disimpan ke data/dataset.txt
```

16. Menu 0: Keluar

```
Pilih menu: 0
```

```
Program selesai.
```

17. Menu selain 0 - 14

```
Pilih menu: 15
```

```
Menu tidak tersedia.
```

I. Analisis Kompleksitas

Operasi	Struktur/Algoritma	Kompleksitas	Alasan
Tampilkan semua checkpoint	LinkedHashMap	$O(V)$	Membuat list baru dari semua checkpoint.
Ambil checkpoint berdasarkan ID	HashMap/LinkedHashMap	$O(1)$ rata-rata	Akses langsung memakai key ID.
Search checkpoint	Linear search	$O(V)$	Mengecek setiap checkpoint satu per satu.
Insert checkpoint	Map + rebuild graph + heap	$O(V + E)$	Setelah insert, struktur graph dan heap dibangun ulang.

Update checkpoint	Map + route scan + rebuild	$O(V + E)$	Mengecek/mengubah checkpoint, memperbarui route terkait, lalu rebuild.
Delete checkpoint	Map + route filter + rebuild	$O(V + E)$	Menghapus checkpoint, menghapus route terkait, lalu rebuild.
Tampilkan struktur graph	Adjacency list	$O(V + E)$	Menelusuri semua vertex dan seluruh edge pada adjacency list.
Ambil route	Linear search edge list.	$O(E)$	Mengecek route satu per satu sampai ditemukan.
Insert route	Edge list + rebuild	$O(V + E)$	Validasi mengecek route, lalu graph dan heap dibangun ulang.
Update route	Edge list + rebuild	$O(V + E)$	Mencari, menghapus, menambah route baru, lalu rebuild.
Delete route	Edge list + rebuild	$O(V + E)$	Menghapus route dari list, lalu rebuild jika route ditemukan.
Tampilkan area terblokir	Linear scan	$O(V + E)$	Mengecek semua checkpoint dan semua route.
BFS	Queue + adjacency list	$O(V + E \log V)$ worst case	Setiap vertex/edge dikunjungi, tetapi neighbor di-sort saat eksplorasi. Tanpa sorting: $O(V + E)$.
A* pathfinding	Binary min heap + graph	$O((V + E) \log E)$	Setiap kandidat jalur masuk heap, operasi insert/extract membutuhkan $O(\log E)$.

A* tanpa heuristic	Dijkstra-like search	$O((V + E) \log E)$	Sama seperti A*, tetapi heuristic bernilai 0.
Bandingkan heuristic	Dua kali A*	$O((V + E) \log E)$	Menjalankan A* dua kali; konstanta 2 diabaikan dalam Big-O.
Ubah bobot terrain	EnumMap	$O(1)$	Jumlah terrain kecil/konstan, update langsung berdasarkan key.
Peek checkpoint paling berbahaya	Max heap	$O(1)$	Elemen maksimum selalu berada di root heap.
Ranking checkpoint berbahaya top-k	Max heap copy + extract	$O(V + k \log V)$	Membuat copy heap $O(V)$, lalu extract max sebanyak k kali.
Simpan dataset	Sorting + file write	$O(V \log V + E \log E)$	Checkpoint dan route diurutkan sebelum ditulis.
Load dataset	File parsing + rebuild	$O(V + E)$	Membaca semua checkpoint dan route, lalu rebuild struktur.
Rebuild graph dan heap	Adjacency list + max heap	$O(V + E)$	Menambahkan semua checkpoint, semua route, lalu heapify checkpoint.
Insert heap minimum	Binary min heap	$O(\log n)$	Elemen naik lewat bubbleUp.
Extract heap minimum	Binary min heap	$O(\log n)$	Root dihapus, elemen terakhir turun lewat bubbleDown.
Rebuild max heap	Max heap	$O(V)$	Heapify dari tengah array ke awal.
Extract max heap	Max heap	$O(\log V)$	Root diganti elemen terakhir, lalu bubbleDown.

J. What-If Analysis

What-if analysis adalah analisis untuk melihat bagaimana hasil algoritma berubah ketika kondisi input atau parameter diubah. Pada project ini, kondisi yang dapat diubah antara lain:

- bobot terrain
- checkpoint yang terblokir
- penggunaan heuristic pada A*

Tujuannya adalah menunjukkan bahwa struktur data dan algoritma yang dipakai tidak hanya berjalan pada satu kondisi tetap, tetapi juga dapat dianalisis perilakunya saat environment berubah.

1. Skenario 1: Perbandingan A* dengan dan tanpa heuristic

Membandingkan pengaruh heuristic Manhattan terhadap efisiensi pencarian jalur.

Input:

- Menu: 11
- Start: A1
- Goal: E5

Hasil:

- A* dengan Manhattan
 - Jalur: A1 -> B1 -> C1 -> D1 -> E1 -> E2 -> E3 -> E4 -> E5
 - Total cost: 10
 - Node diekspansi: 16
- A* tanpa heuristic
 - Jalur: A1 -> B1 -> C1 -> D1 -> E1 -> E2 -> E3 -> E4 -> E5
 - Total cost: 10
 - Node diekspansi: 19

```
A* dengan Manhattan:  
- Jalur: A1 -> B1 -> C1 -> D1 -> E1 -> E2 -> E3 -> E4 -> E5  
- Total cost: 10  
- Node diekspansi: 16  
A* tanpa heuristic (setara Dijkstra):  
- Jalur: A1 -> B1 -> C1 -> D1 -> E1 -> E2 -> E3 -> E4 -> E5  
- Total cost: 10  
- Node diekspansi: 19
```

Pada kondisi ini, heuristic Manhattan tidak mengubah jalur terbaik dan tidak mengubah total cost, sehingga solusi tetap optimal. Namun, heuristic mengurangi jumlah node yang diekspansi dari `19` menjadi `16`. Artinya, heuristic membantu mengarahkan pencarian ke tujuan dengan lebih efisien dibanding pencarian tanpa heuristic.

2. Skenario 2: Perubahan bobot terrain WATER

Melihat apakah perubahan bobot terrain dapat mengubah cost jalur atau bahkan mengubah jalur terbaik.

Input:

- Menu: 12
- Start: A2
- Goal: C4
- Terrain yang diubah: `WATER`

Kondisi awal

- Bobot WATER = 4
- Jalur terbaik: A2 -> A3 -> B3 -> B4 -> C4
- Total cost: 10
- Node diekspansi: 15

Percobaan 1: WATER diturunkan menjadi 1

Hasil:

- Jalur: A2 -> A3 -> B3 -> B4 -> C4
- Total cost: 7
- Node diekspansi: 12

```
Sebelum perubahan:
- Jalur: A2 -> A3 -> B3 -> B4 -> C4
- Total cost: 10
- Node diekspansi: 15
Sesudah perubahan:
- Jalur: A2 -> A3 -> B3 -> B4 -> C4
- Total cost: 7
- Node diekspansi: 12
```

Saat bobot `WATER` diturunkan, jalur terbaik tidak berubah karena jalur tersebut memang sudah memanfaatkan route `WATER` yang menguntungkan. Yang berubah adalah total cost menjadi lebih kecil dan pencarian menjadi sedikit lebih efisien.

Percobaan 2: WATER dinaikkan menjadi 8

Hasil:

- Jalur terbaik: `A2 -> A1 -> B1 -> C1 -> D1 -> D2 -> D3 -> D4 -> C4`

- Total cost: `12`
- Node diekspansi: `19`

Perbandingan Rute:

Sebelum perubahan:

- Jalur: A2 → A3 → B3 → B4 → C4
- Total cost: 7
- Node diekspansi: 12

Sesudah perubahan:

- Jalur: A2 → A1 → B1 → C1 → D1 → D2 → D3 → D4 → C4
- Total cost: 12
- Node diekspansi: 19

Saat bobot `WATER` dinaikkan, jalur awal menjadi lebih mahal sehingga A* memilih rute alternatif yang menghindari terrain `WATER`. Ini menunjukkan bahwa perubahan parameter terrain dapat langsung memengaruhi keputusan pathfinding. Pada kondisi ini, algoritma tidak sekadar menghasilkan cost yang berbeda, tetapi juga memilih lintasan yang berbeda.

3. Skenario 3: Target berada pada area terblokir

Melihat perilaku algoritma ketika tujuan tidak dapat diakses.

Fakta pada dataset:

- Checkpoint `C3 (Forbidden Core)` berstatus blocked
- Route ke `C3` dari arah sekitar juga bertipe `WALL`

Input:

- Menu: 10
- Start: A1
- Goal: C3

Kondisi awal

- Bobot WATER = 4
- Jalur terbaik: A2 → A3 → B3 → B4 → C4
- Total cost: 10
- Node diekspansi: 15

Percobaan 1: WATER diturunkan menjadi 1

Hasil: Path tidak ditemukan.

```
Pilih menu: 10
```

```
Masukkan checkpoint awal: A1
```

```
Masukkan checkpoint tujuan: C3
```

```
Path tidak ditemukan.
```

Pada kondisi ini, algoritma tidak memaksakan pencarian ke node yang memang tidak valid untuk dilalui. Hasil ini sesuai dengan desain sistem, karena checkpoint blocked dan route `WALL` diperlakukan sebagai area impassable. Dengan demikian, sistem dapat membedakan antara jalur mahal dan jalur yang memang tidak boleh dilalui sama sekali.

K. Kesimpulan

Berdasarkan hasil perancangan dan pengujian, sistem Game Map Pathfinding AI berhasil merepresentasikan peta permainan berbobot menggunakan struktur data graph dengan 25 checkpoint dan 43 route. Setiap terrain memiliki bobot berbeda sehingga algoritma dapat menentukan jalur terbaik berdasarkan total cost, bukan hanya jumlah checkpoint yang dilewati.

Algoritma A* berhasil digunakan untuk mencari jalur optimal dari titik awal ke tujuan dengan mempertimbangkan bobot terrain dan heuristic Manhattan. Dari hasil pengujian, penggunaan heuristic tidak mengubah total cost maupun jalur terbaik, tetapi mampu mengurangi jumlah node yang diekspansi, sehingga pencarian menjadi lebih efisien dibandingkan A* tanpa heuristic atau Dijkstra-like search.

Selain itu, algoritma BFS berhasil digunakan untuk eksplorasi map dari checkpoint tertentu, sedangkan struktur data Max-Heap dapat menampilkan checkpoint dengan danger score tertinggi secara efisien. Sistem juga mampu menangani checkpoint blocked dan route bertipe WALL sebagai area yang tidak dapat dilewati.

Melalui fitur perubahan bobot terrain secara dinamis, sistem menunjukkan bahwa perubahan kondisi map dapat memengaruhi total cost bahkan mengubah jalur terbaik yang dipilih. Dengan demikian, project ini membuktikan bahwa kombinasi graph, A*, BFS, dan heap dapat digunakan secara efektif untuk membangun sistem pathfinding sederhana pada game map berbobot.